

5

counting topics

8

exact solvers

1169

automated checks passing

4

team members

[1 Details](#)[2 Team](#)[3 Problem](#)[4 Math](#)[5 Arch.](#)[6 Screens](#)[7 Testing](#)[8 Contrib.](#)[9 Conclusion](#)**1 Project details**[▲ top](#)

Title	Granary Siege: a combinatorics resource-strategy game	Track	A: Gamified Application
Unit / topic	Unit IV: Combinatorics and Recurrence (pigeonhole, inclusion-exclusion, permutations and combinations, stars and bars, derangements)	Platform	Web app: HTML5, CSS3, vanilla JavaScript (ES2020, BigInt for exact arithmetic). No external libraries. Node.js 18+ only for tests.
GitHub			

2 Team members[▲ top](#)

	Roll No.	Name	Role
RS	256100073	Rudra S Sakhare	Math engine and testing lead
SS	256100075	Sanskar N Salvi	Game design and scoring lead
AW	256100074	Aditya A Waghmode	UI and sandbox lead
YI	256100061	Yashraj R Ingavale	Documentation and QA lead

3 Problem statement and objective[▲ top](#)

What it does. Granary Siege is a browser game where a village is defended by solving randomly generated counting problems. Before each round the player wagers grain, then answers with a whole number; every answer is followed by a full step-by-step derivation. A Sandbox Lab accepts any user input, so nothing is hard-coded. **Who uses it.** S.Y. IT students learning counting techniques. **Concepts shown.** Pigeonhole principle, inclusion-exclusion, permutations and combinations, stars and bars, derangements. **Game rules.** Start with 60 grain and 3 walls. Correct answer: gain $R = 15 + 5L + 2w + 2s - h$ (level L , wager $w \in \{0,10,25,50\}$, streak s , hint $h = 5$). Wrong answer: lose w and 1 wall. Every 3 correct answers raise the level; Stars & Bars unlock at level 2, Derangements at level 3. The game ends at 0 walls.

4 Mathematical formulation[▲ top](#)

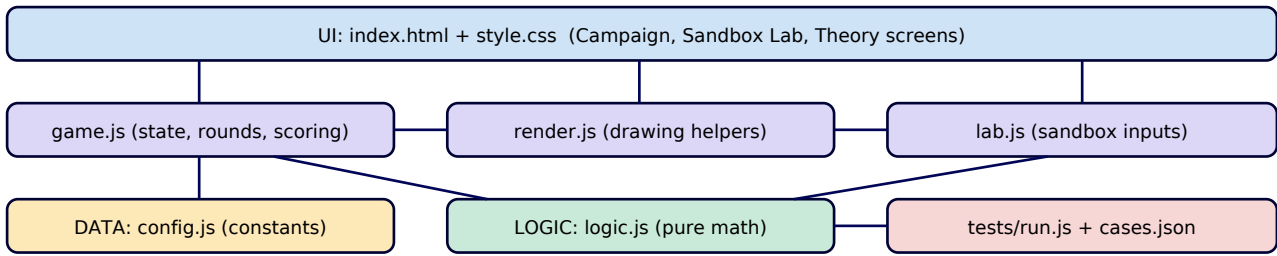
Topic	Definition, notation and rule
Pigeonhole	If N objects are placed in K boxes, then $\exists$ a box holding at least $\lceil N/K \rceil$ objects. To force $\geq r$ in some box, $N = K(r-1)+1$ objects are needed.
Inclusion-Exclusion	$ A \cup B \cup C = A + B + C - A \cap B - A \cap C - B \cap C + A \cap B \cap C $. Members of no set: $ U - A \cup B \cup C $. Consistency conditions: $ A \cap B \leq \min(A , B)$ (same for the other pairs), $ A \cap B \cap C \leq \min$ of the pairwise intersections, every Venn region ≥ 0 , and $ A \cup B \cup C \leq U $.
Selections	$C(n, r) = n! / (r!(n-r)!)$, $P(n, r) = n! / (n-r)!$, for $0 \leq r \leq n$; $C(n, 0) = C(n, n) = 1$ and $C(0, 0) = 1$ (the selection $\emptyset$). Two rivals apart: $C(n, r) - C(n-2, r-2)$.
Stars and bars	Multisets of size r from n types: $C(r+n-1, r)$.
Derangements	$D(n) = n! \sum_{k=0}^n (-1)^k / k!$ and $D(n) = (n-1)(D(n-1) + D(n-2))$, with $D(0) = 1$, $D(1) = 0$.

Algorithm (used for every topic)

- Input and validation:** take parameters from the generator or the user; reject non-integers, negatives, $r > n$, $K = 0$ and impossible Venn sizes with a message.
- Select the rule** for the topic ($\lceil N/K \rceil$, $K(r-1)+1$, inclusion-exclusion, C , P , rivals, stars and bars, $D(n)$).
- Evaluate exactly** with BigInt ($C(n, r)$ multiplies and divides one factor at a time, so every intermediate value is an integer).
- Record each intermediate expression** as a displayed step; for Venn problems also compute every region.
- Compare** the player's integer with the exact answer and apply the scoring rule (grain, walls, streak, level).
- Render** the steps plus the storehouse or Venn diagram.

Before you submit (click to tick):

- ☐ GitHub repo public
- ☐ Screenshots replaced
- ☐ README + tests pushed
- ☐ All members committed
- ☐ Fields saved



logic.js pure counting engine, no DOM code; validates input and returns the answer with steps.

game.js campaign state, random round generator, scoring and level progression.

lab.js sandbox: sends any typed input to logic.js and shows errors clearly.

config.js data: starting grain and walls, wagers, reward constants, unlock levels.

render.js draws step lists, clickable storehouses and the Venn diagram.

index.html, style.css three screens and the cursor-reactive borders.

6 Key UI screenshots

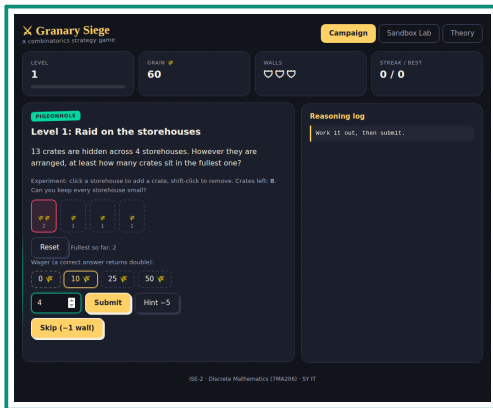


Fig. 1 Input screen: storehouse experiment, wager chosen, answer typed.

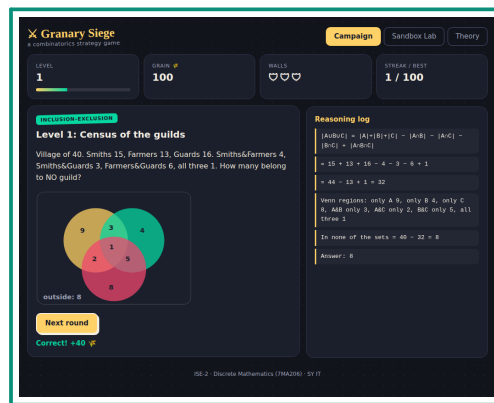


Fig. 2 Output screen: correct answer, step-by-step derivation and Venn regions.

7 Testing and edge cases

9 of 9 report cases passed ((E) = edge case: $\emptyset$ sets, $r > n$, zero boxes). Full suite: **1169 checks** via node tests/run.js, including brute-force cross-checks.

ID	Input	Expected	Actual	Result
T1	Pigeonhole: $N=23$ items, $K=5$ boxes	5	5	PASS
T2	Force $r=6$ in $K=5$ boxes	26	26	PASS
T3	IE: $ A =30$, $ B =25$, $ C =20$; pairs 10, 8, 6; triple 3; $ U =60$	6	6	PASS
T4	$C(10,4)$	210	210	PASS
T5	Rivals: choose 4 of 10, two rivals apart	182	182	PASS
T6	Derangements $D(5)$	44	44	PASS
T7 (E)	IE with $A=B=C=\emptyset$ (all 0), $ U =5$	5	5	PASS
T8 (E)	$C(3,5)$: r greater than n	Rejected	Rejected: r cannot exceed n	PASS
T9 (E)	Pigeonhole with $K=0$ boxes	Rejected	Rejected: Boxes K must be at least 1.	PASS

8 Individual contribution

Member	Built (module or feature) and files / functions
Rudra S Sakhare	Math engine and testing lead. js/logic.js (pigeonFullest, pigeonForce, ie3, choose, perm, rivals, starsBars, derangement, checkInt); tests/run.js, tests/cases.json
Sanskar N Salvi	Game design and scoring lead. js/game.js (makeRound, resolve, submit, nextRound, level and topic unlocking); js/config.js (scoring constants)
Aditya A Waghmode	UI and sandbox lead. js/render.js (drawTrial, staticBins, vennSvg, showSteps); js/lab.js; index.html; style.css (interactive borders)
Yashraj R Ingavale	Documentation and QA lead. README.md; docs/ (build_report.py, VIVA_PREP.md); screenshots/; test review and live-demo script

9 Conclusion and references

Achieved. A complete game plus sandbox covering five counting topics, with exact arithmetic, step-by-step reasoning, input validation and an automated test suite. **Limitation.** Inclusion-exclusion is limited to three sets, and the game is single-player and runs in the browser only. **Future work.** Four-set Venn diagrams, a shared leaderboard, and levels for relations and graph theory. **References.** K. H. Rosen, Discrete Mathematics and Its Applications, McGraw-Hill (chapters on Counting and Advanced Counting Techniques); B. Kolman, R. Busby, S. Ross, Discrete Mathematical Structures, Pearson; MDN Web Docs, “BigInt” (ECMAScript).